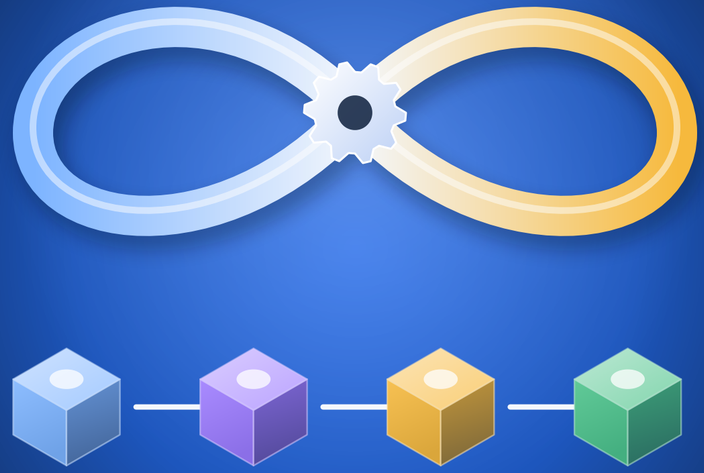
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI5CC · CLOUD COMPUTING · WEEK 12

DevOps and Infrastructure as Code

Deployment on the cloud, CI/CD pipelines and IaC.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Recap

LAST SESSION

Cost management and optimization

Seeing, controlling and reducing what we spend in the cloud.

LAST SESSION

Payment models, pricing model

Pay-as-you-go, save when you commit, pay less by using more.

Overview today

1. Introduction to deployment on cloud with DevOps
2. CI/CD
3. IaC

THIS WEEK

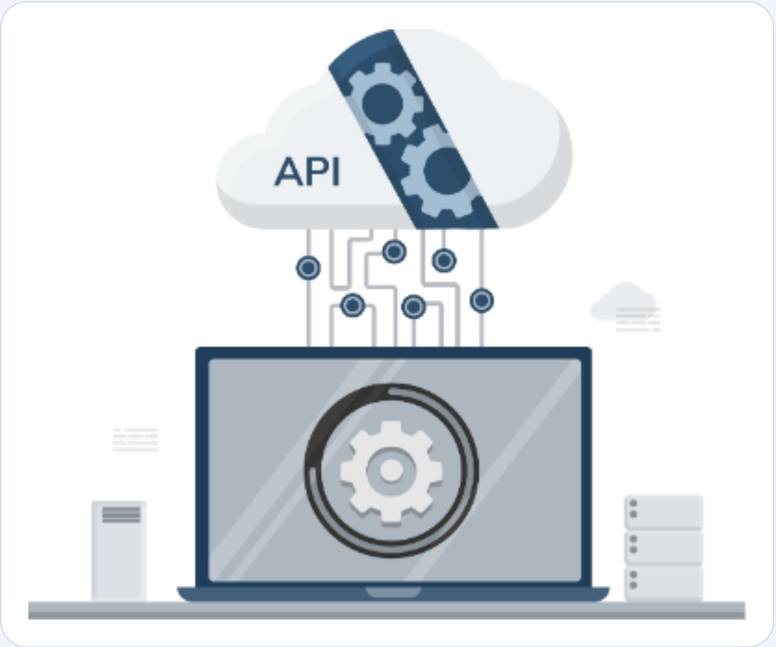
DevOps and IaC

How code and infrastructure reach the cloud, automatically and repeatably.

Introduction to deployment

Deployment is getting hardware and software *up and running* in the desired environment.

It includes steps such as installation, configuration, pushing the code and testing.



Deployment in the cloud

Virtual, not physical

We are not dealing with physical hardware anymore. It is all about the virtual environment or services where we run our applications.

On demand

We can create and destroy resources as needed. To realize the true potential of the cloud, we need to automate as much as possible.

Every service model

IaaS, PaaS and FaaS each approach deployment slightly differently, but the principle remains the same.

For IaaS, we deal with virtual machines or clusters. For PaaS, we provision and use the services without dealing with virtual machines.

PART 1

DevOps

A culture and practice that unifies software development (Dev) and software operation (Ops).

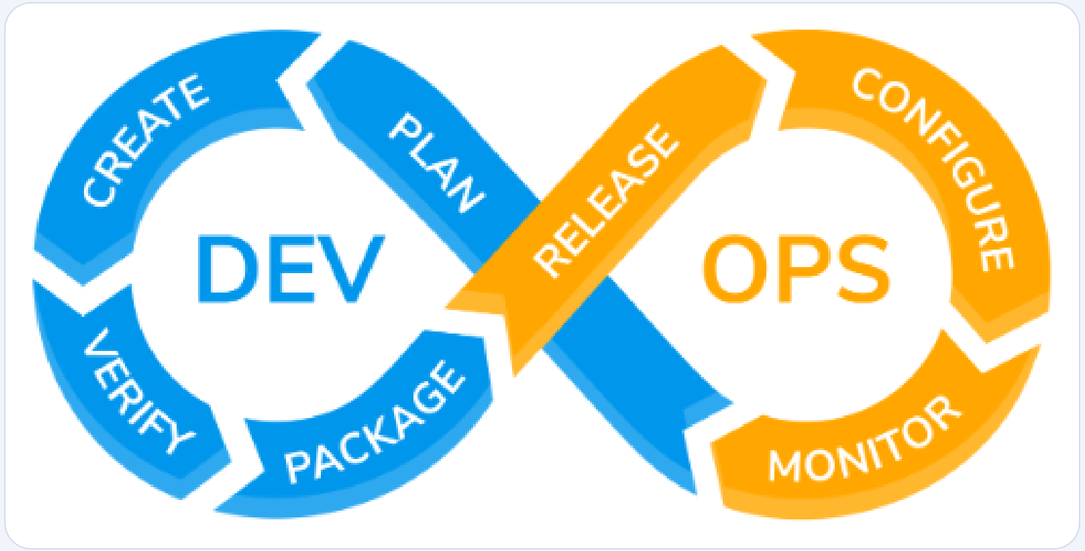
Week 12

What is DevOps?

A software engineering *culture and practice* that unifies Dev and Ops.

It increases an organization's ability to deliver applications and services at high velocity, to better serve customers and compete in the market.

It is a continuous improvement process, often represented as an infinite loop.



The DevOps cycle

Plan, build, test

We start with the planning phase, build a feature, and test for any issues.

Release

Once satisfied with the results, we release the code into the target environment.

Monitor

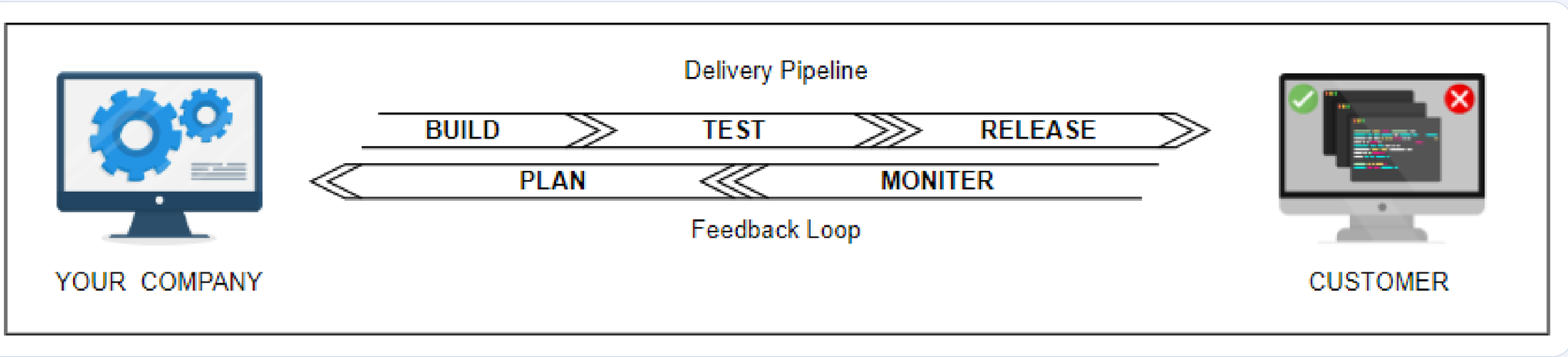
We continuously monitor to detect any issues, and repeat the cycle.

Automate

We use automation and tools to help us during the different phases of the cycle.

The important thing: it is an ongoing process of improvement and collaboration.

DevOps: delivery and feedback



Unlike waterfall, where the whole project is planned first, we plan a feature, implement it, release it, get feedback and keep improving it.

Benefits of DevOps

Speed

Teams release in smaller batches and more often, usually every 2–3 week sprint, instead of one big release every few months.

Reliability

Testing and quality assurance are embedded in the release pipeline, so bugs are detected earlier and fixed before production.

Improved collaboration

Development and operations teams work closely and share the project, so there is less friction between them.

Security

Security audits, security testing and policy as code are built into integration and delivery, so teams deploy with more confidence.

Challenges in DevOps

Shift in mindset

Teams used to work in silos, with a complete architecture in place before any implementation, and long delays before anything reached customers. DevOps breaks down the silos. Teams need training and the opportunity to embrace this way of working, or DevOps remains a series of rituals without real impact.

Transitioning from legacy systems

It is hard to push DevOps practices while teams work with traditional architectures. Breaking big monolithic applications into smaller, independent microservices means learning new things and unlearning old ones. Companies need to provide resources and training, and manage the change.

DevOps services and tools

Provider	Services and tools
AWS	AWS CodePipeline, AWS CodeBuild, AWS CodeDeploy, AWS CodeStar
Azure	Azure DevOps (Azure Repos, Azure Pipelines, Azure Boards, Azure Artifacts)
GCP	Cloud Build, Tekton, Google Cloud Deploy
Atlassian	Bitbucket, Jira, Confluence
Others	GitHub, GitLab, Jenkins, Ansible, Puppet, Chef

DevOps is more about culture and practices than tools, but we still need tools to take advantage of the practices.

PART 2

CI/CD

The practices we follow to realize the DevOps process. Each team can adopt as little or as much as fits.

Continuous integration

A set of practices where development teams implement small changes and check in code to version control repositories frequently.

It establishes a consistent, automated way to build, package and test applications, with workflows that run whenever a change is made to a branch.

What you need	Benefit
Automated tests	Less bugs get shipped
Continuous integration server	Building the release is easy
Developers merge their changes as often as possible	Less context switching
Installation and maintenance of CI server	Reduced testing and QA time

Continuous delivery

Picks up where continuous integration ends. It automates the delivery of applications to environments such as development, test and production.

Tools and automation deploy the code for us. The decision to release a feature in any environment is still manual.

What you need (cost)	Benefit
Strong foundation in continuous integration	Reduced complexity of release
Enough code coverage (tests)	Accelerated release cycle
Automated deployments (manual triggers)	Iterate faster
Feature flags	Reduced testing and QA time

Continuous deployment

All changes that pass integration and delivery are immediately exposed to customers. The decision to release is automated too.

It gives faster access to new features and bug fixes, and quick feedback for developers.

What you need (cost)	Benefit
Strong test culture	Faster development cycle
Up-to-date documentation	Manageable releases
Heavy use of feature flags	Improved quality of solution
Feature flags for unfinished changes	Faster delivery of value to customers

Continuous deployment: the safeguards

We put safeguards and checks in our workflow. As long as those conditions are met, we release to the production environment.



Testing

Automated testing, integration tests and end-to-end tests.



Performance

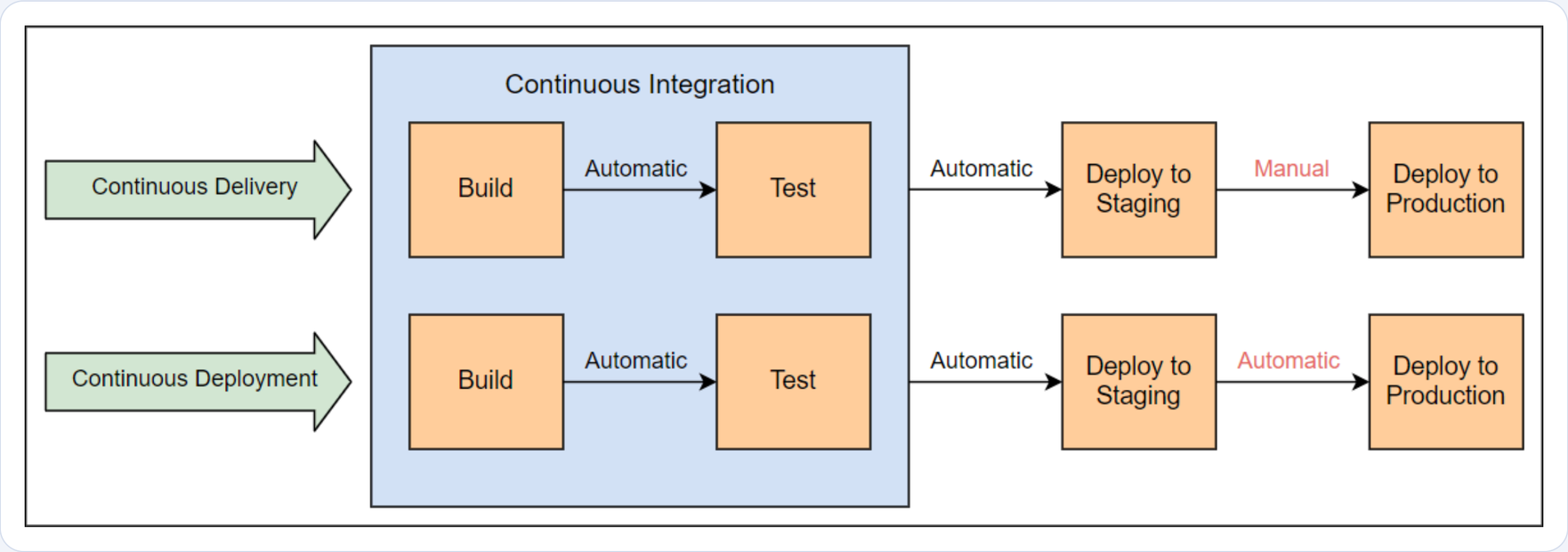
Performance testing.



Monitoring and alerts

Track application health and performance, with alerts set up if there is an issue.

Continuous delivery vs continuous deployment



In continuous delivery, the push to production is manual. In continuous deployment, that step is also automated.

DevOps is a journey

01

Continuous integration

Where many organizations start.

02

Continuous delivery

The next step, once integration is a strong foundation.

03

Continuous deployment

Going all in takes a significant amount of work and maturity.

Every company takes a different path, and this process can take time.

Deployment pipelines

The DevOps pipeline is a set of *tools and automated processes* that build and deploy code to the targeted environment.

Why we need it

Without the necessary tools and automation, it becomes difficult to put DevOps principles into practice.

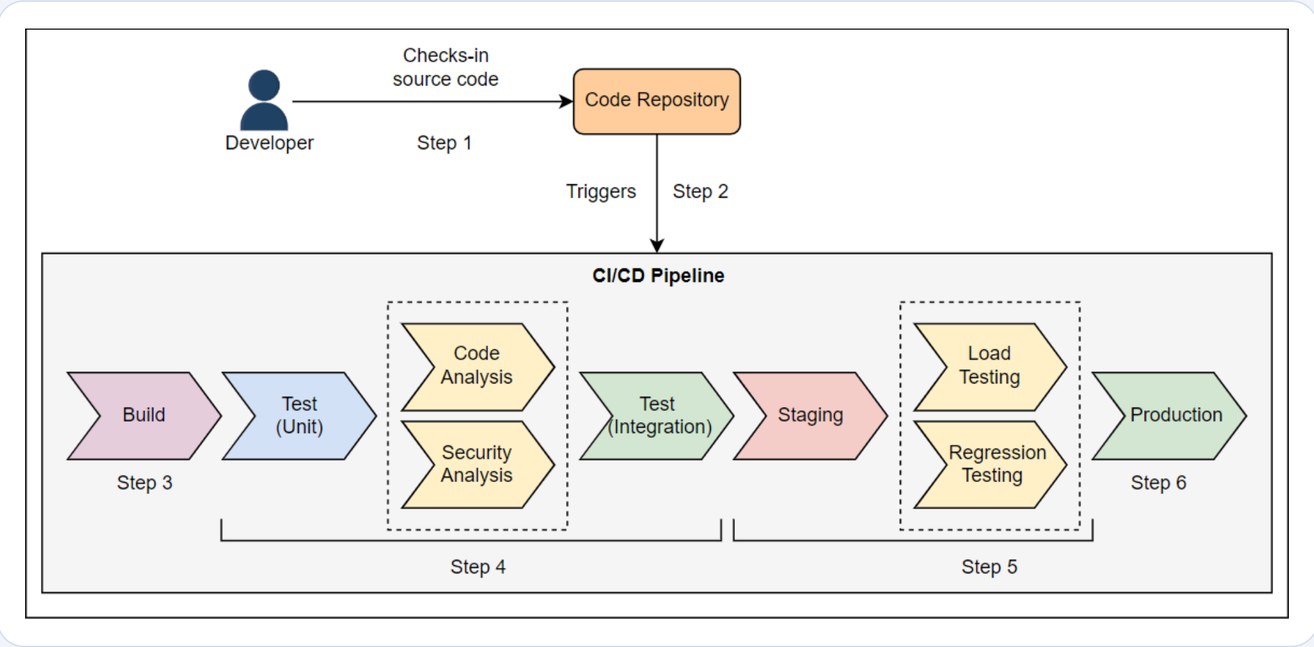
Also called CI/CD

It follows the continuous integration, continuous delivery and continuous deployment processes of DevOps.

It varies

A pipeline differs from one organization or service to another, but usually involves similar steps.

DevOps pipeline flow



A workflow of steps run in sequence or in parallel. The goal: from code to a deployed functionality, with as much automation as we can manage.

DevOps pipeline flow: six steps

01

Check in

A developer checks the source code into a remote repository, the central place for the team's code.

02

Trigger

A new check-in triggers the pipeline. This step can be manual, but is usually automatic.

03

Build

The pipeline pulls the source code from the repository and compiles it.

04

Test and analyze

Unit and integration tests, code and security analyses. Good practice, but up to the team.

05

Staging

Deploy to the staging or QA environment for load, regression and user acceptance tests.

06

Production

Promote the code to production. Manual: continuous delivery. Automated: continuous deployment.

PART 3

Infrastructure as Code

Treat infrastructure the way we treat application code.

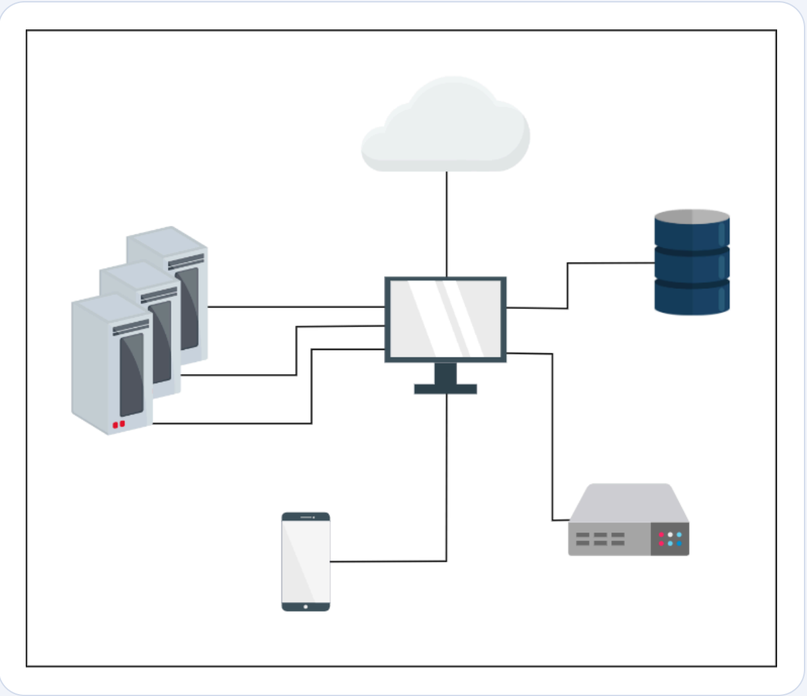
Week 12

Infrastructure as Code

IaC applies *DevOps principles* to managing infrastructure resources.

We define infrastructure in the form of configurations that specify the desired state of the resources we want.

The resources can be virtual machines, networking resources, storage and database resources.



How Infrastructure as Code works

Configurations

Often written in the JSON or YAML format, though there is no defined standard.

Source control and pipelines

We keep configurations in a source control system such as Git, and use CI/CD pipelines to provide the resources.

Desired state

Not only creating, updating or deleting resources, but applying their configuration, so they are in the state needed to function.

Example: after creating a virtual machine, associate a public IP address with it, and restrict which IP addresses can reach it.

Benefits of Infrastructure as Code

Automation

Provisioning is automated, reducing manual effort and making processes faster.

Consistency

Infrastructure is provisioned the same way every time, minimizing errors and deviations.

Version control

Track changes with tools like Git, roll back, and see how infrastructure evolved.

Documentation

The code itself shows exactly what the infrastructure looks like at any point in time.

Cost efficiency

Automating deployment and destruction manages resources better, potentially saving cost.

Risk mitigation

Fewer human errors, such as misconfigured servers, and compliance with defined standards.

Infrastructure as Code platforms and tools

Provider	Platform or service
HashiCorp	Terraform
Pulumi	Pulumi
AWS	AWS CloudFormation, AWS CDK
Azure	Azure ARM, Azure Bicep

Task

Find a small example of using IaC in an application.

1. Show the application overview: frontend, backend, database.
2. Define the infrastructure and resources needed: compute, database, networking, storage.
3. Show how to use Terraform or CloudFormation to achieve it.

Next session

01

AI/ML services in the cloud

02

Capstone project: development